UNER

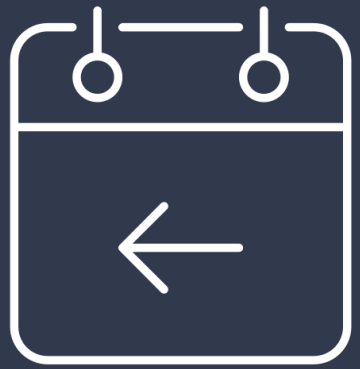
Facultad de Ciencias de la Administración

TECNICATURA UNIVERSITARIA EN DESARROLLO WEB



Desarrollo para dispositivos móviles

Unidad 1 - Introducción



- ## Objetivos
  - Conocer de enfoques de Desarrollo Móvil.
  - Comprender diferentes Frameworks de desarrollo móvil.
- ## Temas a desarrollar
  - Estado Actual de las Aplicaciones Móviles en 2025
  - Panorama de los Enfoques de Desarrollo Móvil
  - Nativas vs. Híbridas vs. Aplicaciones Web: Pros y Contras
  - ¿Qué es un Framework? Framework vs. Librería
  - Frameworks de Desarrollo Móvil No Nativos

La era móvil

Por qué hablamos de Apps



- Los usuarios consumen el doble de internet en su celular que en el escritorio.
- El 90% de ese consumo es a través de Apps.
- Por mes se agregan 100K y 30K nuevas apps a Google Play y Apple Store.
- La Gen-Z abre sus apps más de 50 veces al día.

- Los usuarios prefieren las Apps antes que las páginas Web:
  - Mayor personalización.
  - Notificaciones.
  - Acceso a sensores y dispositivos.
  - Funcionan offline.
  - ...

Referencias

- https://mindsea.com/app-stats/
- https://www.mobilesmith.com/html5-vs-native-debate-is-over/

¿Qué es una app?

Una App o aplicación móvil es un software diseñado para ser instalado en celulares y tablets utilizando el mismo código fuente, solo necesitamos que sea responsive. A diferencia de las aplicaciones web, que son visibles desde el navegador, estas deberán ser descargadas de las tiendas del sistema operativo. Nos permiten acceder a todas las capacidades de los dispositivos.

- Nos permite interactuar con el hardware de los dispositivos:
  - Acceder a la cámara.
  - realizar llamadas.
  - Conocer la ubicación.
  - Transmitir información (bluetooth).

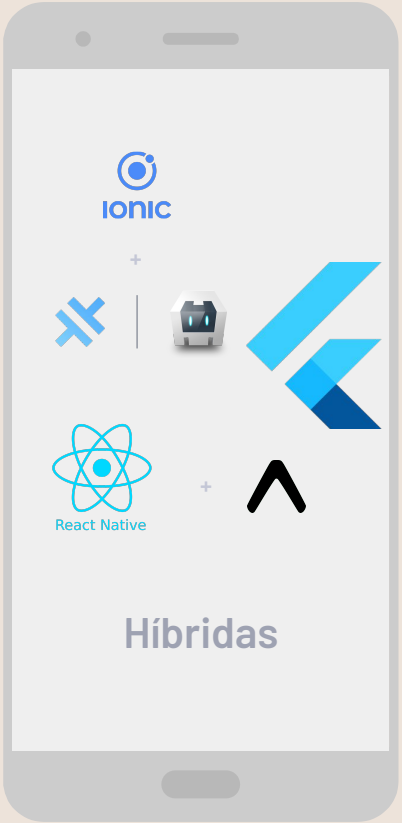
A tener en cuenta!! desarrollar una aplicación requiere mucha planeación, mantenimiento, pruebas (sobre todo para soportar diferentes dispositivos) y trabajo para publicarla. Por estos motivos y otros que vamos durante la cursada hay que hacer un buen análisis para ver si vale la pena o no desarrollar una aplicación para dispositivos móviles o una web.

Estado Actual de las Aplicaciones Móviles

- **Crecimiento del Mercado**: El mercado de aplicaciones móviles alcanzará $613 mil millones para 2027, con 2.17 mil millones de suscripciones 5G en 2024, permitiendo apps más rápidas y complejas.
- **Integración de IA**: El 85% de las interacciones con apps en 2025 involucrarán IA, potenciando experiencias hiperpersonalizadas y chatbots.
- **Dominio Multiplataforma**: Herramientas como React Native y Flutter permiten apps de un solo código, con 11% de desarrolladores usando React Native.
- **Crecimiento de AR/VR**: El mercado de realidad aumentada alcanzará $13.8 mil millones en 2025, mejorando apps en juegos, comercio y empresas.

Enfoques de desarrollo Móvil







Enfoques de Desarrollo Móvil

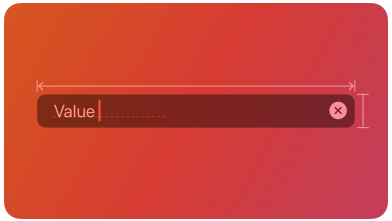
- Las aplicaciones móviles se desarrollan en tres enfoques: Nativas, Híbridas y Aplicaciones Web (Aplicaciones Web Progresivas - PWAs).
- Cada enfoque equilibra rendimiento, costo y velocidad de desarrollo.
- En 2025, frameworks multiplataforma como React Native son populares por su rentabilidad, usados por marcas como Instagram.

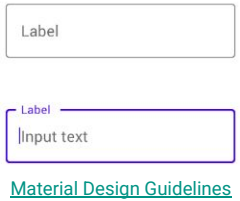
¿Que son?

Significa que el desarrollador va a tener que crear un código fuente para cada una de las plataformas en las cuáles quiera publicar su aplicación, usando el lenguaje de programación definido por la plataforma, herramientas y entornos de desarrollo.

	iOS	Android
Lenguaje	Swift, Objective-C, C#	Java, Kotlin
IDEs	Xcode	Android Studio

- **Pros:**
  - Rendimiento superior con acceso optimizado al hardware, mejorado por la baja latencia de 5G (1ms vs. 50ms de 4G).
  - Interfaz/UX nativa alineada con guías de diseño de la plataforma (p. ej., Material Design, Human Interface Guidelines).
  - Acceso completo a APIs del dispositivo (cámara, GPS, etc.).
  - Fuerte soporte de comunidades de Apple/Google.
- **Contras:**
  - Bases de código separadas aumentan tiempo y costo de desarrollo.
  - Las actualizaciones requieren aprobación de tiendas de aplicaciones, retrasando el despliegue.
  - Costos más altos por desarrolladores especializados.
  - Ejemplo: Aplicaciones como Spotify o Google Maps.





Material Design Guidelines

¿Que son?

Como dice la palabra son un híbrido, una mezcla, entre una aplicación nativa y una web.

Las funcionalidades de la aplicación están escritas con tecnologías web (JS, CSS, HTML), que en lugar de ser mostrada al usuario en un navegador es encapsulada dentro de una aplicación nativa con un navegador embebido en el mismo código nativo transparente para el usuario. En iOS el elemento se llama ***WKWebView*** y en Android ***WebView***. Y todo esto es transparente a los desarrolladores gracias a tecnologías como ***Apache Cordova*** o ***Ionic con Capacitor***

En el caso de ***React Native*** el Framework se encarga de transformar los componentes de React Native en componentes Nativos en tiempo de build.

Estas tecnologías tienen un sistema de plugins que permiten extender las capacidad más allá de lo que el navegador pueda ofrecer.

En el caso de ***React Native*** el Framework se encarga de transformar los componentes de React Native en componentes Nativos en tiempo de build.

Componente React Native

```
<TextInput
    style={styles.input}
    onChangeText={onChangeNumber}
    value={number}
    placeholder="useless placeholder"
    keyboardType="numeric"
/>
```

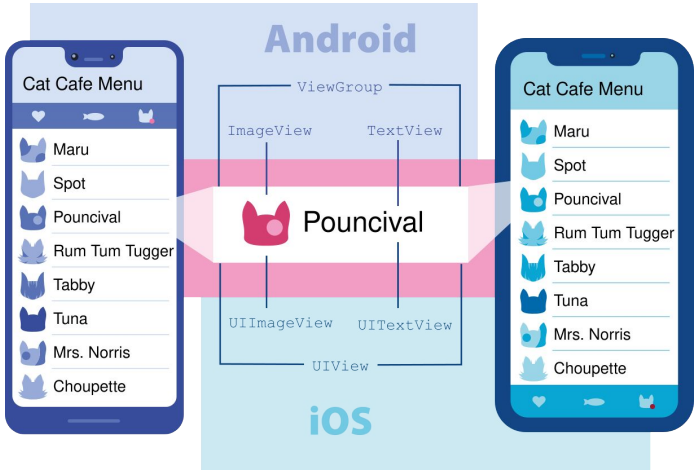
Vista Android

```
<EditText
android:id="@+id/plain_text_input"
android:layout_height="wrap_content"
android:layout_width="match_parent"
android:inputType="text"/>
```

Component iOS

```
@State private var username: String = ""
@FocusState private var emailFieldIsFocused: Bool = false
var body: some View {
    TextField(
        "User name (email address)",
        text: $username
    )
    .focused($emailFieldIsFocused)
    .onSubmit {
        validate(name: username)
    }
    .textInputAutocapitalization(.never)
    .disableAutocorrection(true)
    .border(.secondary)

    Text(username)
    .foregroundColor(emailFieldIsFocused ? .red : .blue)
```



- **Pros:**
  - Una sola base de código reduce el tiempo de desarrollo.
  - Aprovecha habilidades de desarrollo web para prototipado rápido.
  - Rentable comparado con el desarrollo nativo.
  - Algunos frameworks permiten actualizaciones sin aprobación de tiendas, como es el caso de Ionic y React Native+Expo.
- **Contras:**
  - Rendimiento más lento debido a capas de abstracción (p. ej., puente de JavaScript).
  - La interfaz puede no sentirse completamente nativa.
  - Acceso limitado a algunas funciones del dispositivo sin plugins.
  - Dependencia de actualizaciones de frameworks de terceros.

Aplicaciones Web Progresivas - PWAs

Sitios web optimizados para móviles que se ejecutan en navegadores, mejorados con funciones de PWA como, acceso sin conexión, notificaciones push, entre otras.

- **Pros:**
  - Independientes de la plataforma, funcionan en cualquier dispositivo con navegador.
  - Bajo costo de desarrollo usando tecnologías web estándar.
  - Actualizaciones instantáneas sin aprobación de tiendas.
  - No requieren instalación, accesibles vía URLs.
- **Contras:**
  - Acceso limitado al hardware del dispositivo comparado con nativas/híbridas.
  - Rendimiento más lento para apps complejas, incluso con mejoras de 5G.
  - Experiencia de usuario varía según navegadores/dispositivos.
  - Menor visibilidad en comparación con tiendas de aplicaciones.

Nativas

Android App

iOS App

Híbridas y Cross Platform

Buildeo nativo

Android App

iOS App

PWA

~~Tecnologías Web~~



Construyendo una app

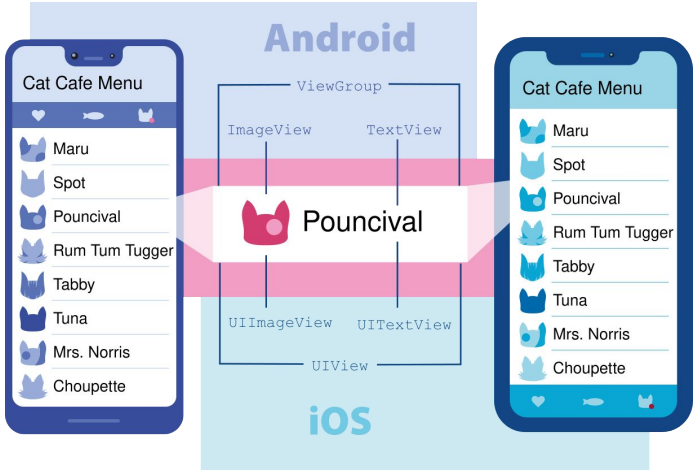
Cross Platform





Buildeo nativo





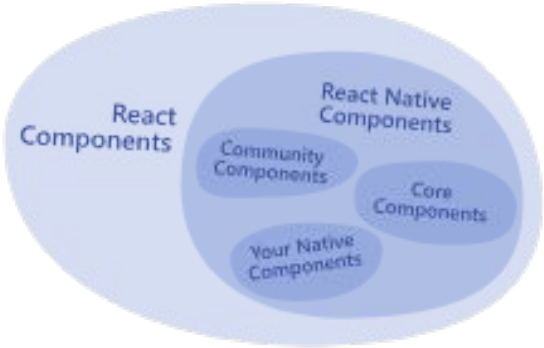


Tabla comparativa

Aspecto	Nativo	Híbrido	PWA
Costo de Desarrollo	Alto (bases de código duales)	Moderado (base de código única)	Bajo (tecnologías web)
Interfaz/UX	Específica de la plataforma	Casi nativa	Dependiente del navegador
Func. de Dispositivo	Acceso completo a APIs	Limitado (vía plugins)	Restringido
Actualizaciones	Requiere aprobación de tiendas	Tiendas/sin Tiendas	Instantáneas (web)
Ejemplos	Spotify, Google Maps	Instagram, Uber	Twitter Lite, Starbucks

Framework

También conocido como marco de trabajo, es un conjunto de herramientas, bibliotecas/librerías con patrones de diseño y convenciones predefinidas que ayudan a desarrollar software de una manera más rápida, ordenada y eficientemente.

Qué hace un Framework:

- **Proporciona una estructura:**
  - Define la organización general del proyecto, incluyendo la estructura de carpetas, archivos y clases.
- **Ofrece componentes reutilizables:**
  - Incluye bibliotecas y módulos para tareas comunes como manejo de datos, seguridad y autenticación.
- **Establece patrones de diseño:**
  - Sugiere patrones de diseño para estructurar el código, promoviendo la consistencia y mantenibilidad.
- **Simplifica tareas repetitivas:**
  - Automatiza tareas comunes como la gestión de sesiones, la interacción con bases de datos, etc.
- **Mejora la productividad:**
  - Permite a los desarrolladores concentrarse en la lógica específica de la aplicación, acelerando el desarrollo.
- **Facilita la colaboración:**
  - Al estandarizar el código y la estructura, facilita el trabajo en equipo.

Librería:

Conjunto de código pre-escrito y reutilizable que facilita el desarrollo de software al proporcionar funciones, clases y otros recursos para realizar tareas específicas

- **Framework:**
  - Controla el flujo y la estructura de la aplicación (inversión de control).
  - Los desarrolladores trabajan dentro de sus reglas.
  - Ejemplo: React Native dicta cómo se renderizan los componentes y se maneja el estado.
- **Librería:**
  - Colección de funciones/clases reutilizables llamadas por los desarrolladores.
  - Más flexible, integrada en la estructura definida por el desarrollador.
  - Ejemplo: Lodash proporciona funciones de utilidad sin imponer estructura.

Frameworks de desarrollo móvil NO nativos

- Una sola base de código para múltiples plataformas.
- Aprovecha habilidades existentes (p. ej., JavaScript para React Native).
- Desarrollo más rápido y costos menores.

React Native (con Expo)

React Native es un framework de código abierto desarrollado por Facebook para construir aplicaciones móviles utilizando JavaScript y React. Permite a los desarrolladores crear aplicaciones móviles nativas para iOS y Android utilizando una sola base de código, aprovechando la arquitectura basada en componentes de React. A diferencia del desarrollo web tradicional con React, React Native se compila a código nativo*, lo que permite que las aplicaciones alcancen un rendimiento cercano al de aplicaciones completamente nativas escritas en Swift o Kotlin, mientras mantiene la flexibilidad de JavaScript. Mientras que Expo es una plataforma de código abierto que simplifica el desarrollo, la construcción y el despliegue de aplicaciones React Native. Proporciona un conjunto de herramientas, servicios y un flujo de trabajo gestionado para facilitar la creación de aplicaciones

- **Pros:**
  - Rendimiento casi nativo.
  - Recarga en caliente para desarrollo rápido.
  - Flujo de trabajo gestionado por Expo simplifica la configuración.
  - Gran comunidad y ecosistema (p. ej., paquetes npm).
- **Contras:**
  - Ligeramente más lento que aplicaciones nativas.
  - Mayor tamaño de la app debido al runtime de JavaScript.
  - Funciones nativas avanzadas pueden requerir módulos personalizados.

Ionic Framework





+



¿Qué es Ionic?

Es una herramienta open source de UI que permite construir aplicaciones móviles y de escritorio utilizando tecnologías web (HTML, CSS y JavaScript) encapsulando el build en webviews nativas, y que además se integra a otros frameworks populares como Angular y Vue o librerías como React

¿Cuándo surgió?

Fue creado en 2013 por 3 desarrolladores de la empresa Drifty Co.

Se construyó sobre AngularJS (Angular 1 y Apache Cordova).

¿Qué es Capacitor?

Es una plataforma open source que permite a los desarrolladores producir apps con tecnologías web. Consiste en en un SDK, una línea de comando (CLI), Plugins para interactuar con los dispositivos y una API para crearlos.

- **Pros:**
  - Fuerte integración con flujos de trabajo de desarrollo web.
  - Amplia librería de plugins para funciones nativas (vía Capacitor).
  - Familiar para desarrolladores web.
- **Contras:**
  - Rendimiento más lento debido a WebView, menos optimizado para apps intensivas en 5G.
  - Interfaz menos nativa comparada con React Native.
  - Dependencia de actualizaciones de WebView.

Flutter

Flutter es un framework de código abierto desarrollado por Google para construir aplicaciones multiplataforma con una sola base de código. Permite crear aplicaciones nativas compiladas para iOS, Android, web, escritorio (Windows, macOS, Linux) y dispositivos integrados, utilizando el lenguaje de programación Dart. A diferencia de frameworks como Ionic + Capacitor, que dependen de WebViews, o React Native, que usa un puente JavaScript, Flutter compila directamente a código nativo*, ofreciendo un rendimiento cercano al de aplicaciones nativas.

*Compila directamente el código Dart a código de máquina nativo (ARM o x86) usando el motor gráfico Skia, sin depender de un puente. Toda la UI y la lógica se renderizan directamente en el dispositivo, lo que elimina la sobrecarga de comunicación entre JavaScript y nativo, ofreciendo un rendimiento generalmente más fluido para casos complejos.



Flutter

Flutter - Características

- **Interfaz de usuario basada en widgets:** Flutter utiliza un sistema de widgets altamente personalizables para construir interfaces de usuario declarativas, con componentes predefinidos (Material Design y Cupertino) que se adaptan a las plataformas.
- **Rendimiento nativo:** Compila a código de máquina nativo (ARM o x86) usando el motor Skia para gráficos, eliminando la necesidad de puentes o WebViews.
- **Hot Reload:** Permite a los desarrolladores ver cambios en tiempo real durante el desarrollo, acelerando la iteración.
- **Dart como lenguaje:** Dart es un lenguaje optimizado para UI, con sintaxis clara y soporte para programación reactiva. Muy similar a Java, Javascript, or any other C-like language.
- **Soporte multiplataforma:** Una sola base de código para móviles, web, escritorio y dispositivos integrados, con plugins para acceder a funciones nativas como cámara, GPS y notificaciones.
- **Ecosistema de paquetes:** Pub.dev ofrece una amplia gama de paquetes para extender funcionalidades, como integración con Firebase o animaciones avanzadas.

[Bibliografía]

- https://ionic.io/blog/capacitor-everything-youve-ever-wanted-to-know
- Mahagamage, L. R. (2023). Ultimate Flutter Handbook: Learn Cross-Platform App Development with Visually Stunning UIs and Real-World Projects. Orange Education Pvt Ltd. ISBN-10: 9388590864.
- Boduch, A., & Sakhniuk, M. (2024). React and React Native: Build Cross-Platform JavaScript and TypeScript Apps for the Web, Desktop, and Mobile (5th ed.). Packt Publishing. ISBN-10: 1805127306.
- Andreas Dormann. (2023). Ionic 7: Crear aplicaciones increíbles y habilitadas para IA en cualquier plataforma. BPB Publications. ISBN-10: 3945102596.